hook – *is* the pointer to the hook function of type *nf_hookfn*. We populate this field with the function *hook_fn_icmp*.

–is the protocol family identifier. *PF_INET* for IPv4 and *PF_INET6* for IPv6.

hooknum – identifies the hook for the hook function pointed to by the hook field. We populate this field with *PF_INET_PRE_ROUTING* because the *hook_fn_icmp* has to filter and drop the ICMP packets just as they arrive.

priority – is a module that may register multiple hook functions at one hook. Hence, this field specifies the priority in which the hook function registered at the hook is called. The smaller the value, the higher is the priority. For IPv4, the priorities are defined in *netfilter_ipv4.h as enum nf_hook_priorities*. In our implementation, we assign the member with *NF_IP_PRI_FIRST* – assigning the highest priority for our hook function *nf_hook_icmp*.

3) **Registering the *nf_hook_ops* structure**: Finally, we register a properly initialised *nf_hook_ops* structure in the *init* function (*fw_init()* of our kernel module (at Line 48) via the following function:

```
nf_register_hook(struct nf_hooks_ops *);
```

Similarly, we unregister the *nf_hook_ops* structure in the exit function (at Line 56) using the following command:

```
nf_unregister_hook(struct nf_hooks_ops *) ;
```

The prototypes of both the functions are available in *<linux/netfilter.h>:*

The source: icmp_drop.c

```
 1 /* icmp_drop.c - firewall using Netfilter to block ICMP
ping packets from 192.168.164.51 */
 2
 3 #include <linux/ip.h>
 4 #include <linux/kernel.h>
 5 #include <linux/module.h>
 6 #include <linux/netdevice.h>
 7 #include <linux/netfilter.h>
 8 #include <linux/netfilter_ipv4.h>
 9 #include <linux/skbuff.h>
10 #include <linux/udp.h>
11 #include <linux/if_ether.h>
12
13 MODULE_AUTHOR("roy");
14 MODULE_LICENSE("GPL");
15
16 static struct nf_hook_ops nf_ops_pre_route;
       /* nf_hook_ops struct at PRE_ROUTING hook */
17 static unsigned char *ip_address = "\xC0\xA8\xA4\x33";
     /* 192.168.164.51 */
18 static int drop_count = 0;
        /* dropped pkts count */
19 struct sk_buff  *sock_buff;
     /* pointer to kernel socket buffer */
20 struct iphdr    *ip_header;
/* pointer to ip header of the pkt */
21
22 /* hook function at PRE_ROUTING hook */
23 unsigned int hook_fn_icmp(unsigned int hooknum, struct
sk_buff *skb,
24                 const struct net_device *in,const struct
net_device *out,
25                 int (*okfn)(struct sk_buff*))
26 {
27     sock_buff = skb;
28
29     ip_header = (struct iphdr *)skb_network_header(skb);
30
31     if (ip_header->saddr == *((unsigned int *)ip_address)
32                 && (ip_header->protocol==0x01)) /* check
for src ip 192.168.164.51 and icmp pkts*/
33     {
34         printk(KERN_ALERT"NF_INET_PRE_ROUTING : Dropping
ICMP packet# %d\n",                 ++drop_count);
35         return NF_DROP;     /* Drop the ICMP packet */
36     }
37     return NF_ACCEPT;       /* ACCEPT  rest of the pkts
*/
38 }
39
40 /* Initialize and register the nf_hook_ops structure in
init function */
41 static int __init  icmp_fwall_init(void)
42 {
43     printk(KERN_ALERT"Loading icmp_drop.ko\n");
44     nf_ops_pre_route.hook       = hook_fn_icmp;
45     nf_ops_pre_route.pf             = PF_INET;
46     nf_ops_pre_route.hooknum  = NF_INET_PRE_ROUTING;
47     nf_ops_pre_route.priority = NF_IP_PRI_FIRST;
48     nf_register_hook(&nf_ops_pre_route);
49     return 0;
50 }
51
52 static void __exit icmp_fwall_exit(void)
53 {
54     printk(KERN_ALERT"Unloading icmp_drop.ko\n");
55     printk(KERN_ALERT"Total ICMP packets dropped:
%d\n",drop_count);
56     nf_unregister_hook(&nf_ops_pre_route);  /* Unregister
the nf_hook_ops structure */
57 }
58
59 module_init(icmp_fwall_init);
60 module_exit(icmp_fwall_exit);
```